

29 DE SEPTIEMBRE DE 2026



ACTIVIDAD INVESTIGATIVA

HERENCIA, POLIMORFISMO Y ENCAPSULAMIENTO

INSTRUCTORA: ALEXANDRA MUÑOZ

APRENDIZ: MICHEL CHANTRE

ADSO - CTPI

Contenido

INTRODUCCIÓN	3
ACTIVIDAD 1 – INVESTIGACIÓN SOBRE HERENCIA	3
1. ¿Qué es la herencia en Programación Orientada a Objetos?	3
2. ¿Para qué sirve?	3
3. ¿Qué es una clase padre?	4
4. ¿Qué es una clase hija?	4
5. ¿Qué elementos puede heredar una clase hija?	4
6. ¿Qué ventajas tiene la herencia?	4
7. ¿Cómo se implementa la herencia en JavaScript?	5
8. ¿Para qué sirven las palabras reservadas extends y super?	5
ANÁLISIS DEL EJEMPLO	6
¿Cuál sería la clase padre?	6
¿Cuáles serían las clases hijas?	6
¿Qué características podrían compartir?	7
¿Qué características serían propias de cada clase?	7
ACTIVIDAD 2 – INVESTIGACIÓN SOBRE POLIMORFISMO	7
9. ¿Qué es el polimorfismo?	7
10. ¿Qué significa polimorfismo en Programación Orientada a Objetos?	7
11. ¿Cómo puede un mismo método tener comportamientos diferentes?	7
12. ¿Qué relación tiene el polimorfismo con la herencia?	8
13. ¿Qué ventajas tiene?	9
14. ¿Cómo se puede implementar en JavaScript?	9
ANÁLISIS DEL EJEMPLO	10
¿El método tiene el mismo nombre?	10
¿El resultado es igual?	10
¿Qué cambia?	10
¿Qué concepto representa esta situación?	11
¿Por qué puede ser útil?	11
ACTIVIDAD 3 – INVESTIGACIÓN SOBRE ENCAPSULAMIENTO	11

15. ¿Qué es el encapsulamiento?	11
16. ¿Para qué sirve?	11
17. ¿Por qué es importante controlar el acceso a los datos?	11
18. ¿Qué problemas pueden presentarse si cualquier parte del programa puede modificar directamente los datos?	11
19. ¿Cómo se puede implementar encapsulamiento en JavaScript?	12
20. ¿Qué significa utilizar # para declarar una propiedad?	12
21. ¿Qué ventajas tiene utilizar propiedades privadas?	13
ANÁLISIS DE LA SITUACIÓN	13
¿Debería cualquier parte del programa modificar directamente el saldo?	13
¿Qué problemas podrían ocurrir?	13
¿Cómo podría protegerse el saldo?	13
¿Qué métodos podrían utilizarse para modificarlo?	13
ACTIVIDAD 4 – CUADRO COMPARATIVO	14
ACTIVIDAD 5 – EJEMPLO DE LA VIDA REAL	14
Herencia: ¿Quién sería la clase padre y cuáles serían las clases hijas?	15
Polimorfismo: ¿Qué comportamiento podría cambiar dependiendo del objeto?	15
Encapsulamiento: ¿Qué información debería estar protegida?	16
ACTIVIDAD 6 – EJERCICIO PRÁCTICO EN JAVASCRIPT	16
CONCLUSIONES	18
FUENTES CONSULTADAS	18

INTRODUCCIÓN

La Programación Orientada a Objetos permite organizar un programa mediante clases y objetos. Dentro de este enfoque existen conceptos importantes como la herencia, el polimorfismo y el encapsulamiento.

La **herencia** permite crear una clase nueva a partir de otra clase y reutilizar sus características. El **polimorfismo** permite que un mismo método tenga diferentes comportamientos dependiendo del objeto que lo utilice. El **encapsulamiento** permite proteger los datos de una clase y controlar la manera en que pueden ser consultados o modificados.

En esta actividad se investigan estos tres conceptos y se presentan ejemplos sencillos en JavaScript para comprender cómo pueden utilizarse en la Programación Orientada a Objetos.

ACTIVIDAD 1 – INVESTIGACIÓN SOBRE HERENCIA

1. ¿Qué es la herencia en Programación Orientada a Objetos?

La herencia es un mecanismo de la Programación Orientada a Objetos que permite que una clase nueva pueda utilizar características y comportamientos de otra clase.

Por ejemplo, una clase Perro puede heredar características de una clase Animal.

2. ¿Para qué sirve?

La herencia sirve principalmente para **reutilizar código** y evitar escribir nuevamente características que varias clases necesitan.

También permite organizar mejor las clases de un programa.

3. ¿Qué es una clase padre?

Una clase padre es la clase que contiene características y métodos que pueden ser heredados por otras clases.

Por ejemplo:

```
class Animal {  
}
```

En este caso, Animal puede ser una clase padre.

4. ¿Qué es una clase hija?

Una clase hija es una clase que hereda características y comportamientos de una clase padre.

Por ejemplo:

```
class Perro extends Animal {  
}
```

Aquí Perro es la clase hija y Animal es la clase padre.

5. ¿Qué elementos puede heredar una clase hija?

Una clase hija puede utilizar propiedades y métodos definidos en la clase padre, según las reglas de acceso de JavaScript.

Por ejemplo, puede heredar información como:

- Nombre.
- Edad.
- Métodos.
- Comportamientos.

6. ¿Qué ventajas tiene la herencia?

Algunas ventajas son:

- Permite reutilizar código.
- Evita repetir características.
- Facilita organizar las clases.
- Permite crear clases más específicas.
- Facilita el mantenimiento del programa.

7. ¿Cómo se implementa la herencia en JavaScript?

En JavaScript se utiliza la palabra reservada `extends`.

Ejemplo:

```
class Animal {  
  comer() {  
    console.log("El animal está comiendo");  
  }  
}
```

```
class Perro extends Animal {  
}
```

La clase `Perro` hereda de `Animal`.

8. ¿Para qué sirven las palabras reservadas `extends` y `super`?

extends

`extends` se utiliza para indicar que una clase hereda de otra.

Ejemplo:

```
class Perro extends Animal {  
}
```

super

super permite utilizar el constructor de la clase padre desde la clase hija.

Ejemplo:

```
class Animal {  
    constructor(nombre) {  
        this.nombre = nombre;  
    }  
}
```

```
class Perro extends Animal {  
    constructor(nombre, raza) {  
        super(nombre);  
        this.raza = raza;  
    }  
}
```

En este caso, `super(nombre)` llama al constructor de `Animal`.

ANÁLISIS DEL EJEMPLO

¿Cuál sería la clase padre?

La clase padre sería:

`Animal`

Porque contiene características que pueden compartir los diferentes animales.

¿Cuáles serían las clases hijas?

Las clases hijas serían:

Perro

Gato

¿Qué características podrían compartir?

Podrían compartir:

- Nombre.
- Edad.
- Método comer().

¿Qué características serían propias de cada clase?

Perro:

- Método ladrar().

Gato:

- Método maullar().

ACTIVIDAD 2 – INVESTIGACIÓN SOBRE POLIMORFISMO

9. ¿Qué es el polimorfismo?

El polimorfismo es un concepto de la Programación Orientada a Objetos que permite que un mismo método tenga diferentes comportamientos dependiendo del objeto que lo utilice.

10. ¿Qué significa polimorfismo en Programación Orientada a Objetos?

Significa que diferentes objetos pueden responder de diferentes maneras ante el mismo método.

Por ejemplo, varios animales pueden tener un método llamado:

hacerSonido()

Pero cada animal puede producir un sonido diferente.

11. ¿Cómo puede un mismo método tener comportamientos diferentes?

Una clase hija puede redefinir un método que recibió de su clase padre.

Ejemplo:

```
class Animal {  
    hacerSonido() {  
        console.log("Sonido de animal");  
    }  
}
```

```
class Perro extends Animal {  
    hacerSonido() {  
        console.log("Guau guau");  
    }  
}
```

```
class Gato extends Animal {  
    hacerSonido() {  
        console.log("Miau miau");  
    }  
}
```

Aunque el método se llama igual, el resultado cambia dependiendo del objeto.

12. ¿Qué relación tiene el polimorfismo con la herencia?

El polimorfismo puede utilizarse junto con la herencia cuando una clase hija modifica o redefine un método de la clase padre.

Por ejemplo:

Animal

|

+--- Perro

|

+--- Gato

Los tres pueden tener un método llamado:

hacerSonido()

Pero cada uno puede tener un comportamiento diferente.

13. ¿Qué ventajas tiene?

El polimorfismo permite:

- Utilizar el mismo nombre de método para diferentes comportamientos.
- Organizar mejor el código.
- Hacer que las clases sean más flexibles.
- Evitar crear métodos con nombres diferentes para acciones similares.

14. ¿Cómo se puede implementar en JavaScript?

Se puede implementar creando un método en una clase padre y redefiniéndolo en las clases hijas.

Ejemplo:

```
class Animal {  
  hacerSonido() {  
    console.log("Sonido");  
  }  
}
```

```
class Perro extends Animal {  
  hacerSonido() {
```

```
        console.log("Guau guau");  
    }  
}
```

```
class Gato extends Animal {  
    hacerSonido() {  
        console.log("Miau miau");  
    }  
}
```

ANÁLISIS DEL EJEMPLO

¿El método tiene el mismo nombre?

Sí.

El método se llama:

hacerSonido()

en los diferentes objetos.

¿El resultado es igual?

No.

El perro produce:

Guau guau

y el gato produce:

Miau miau

¿Qué cambia?

Cambia el comportamiento del método dependiendo del objeto.

¿Qué concepto representa esta situación?

Representa el **polimorfismo**.

¿Por qué puede ser útil?

Porque permite utilizar un mismo método para representar una acción general, pero cada objeto puede realizarla de una manera diferente.

ACTIVIDAD 3 – INVESTIGACIÓN SOBRE ENCAPSULAMIENTO

15. ¿Qué es el encapsulamiento?

El encapsulamiento consiste en proteger los datos internos de una clase y controlar la forma en que otras partes del programa pueden acceder a ellos.

16. ¿Para qué sirve?

Sirve para proteger la información y evitar que otros elementos del programa puedan modificarla directamente de una manera incorrecta.

17. ¿Por qué es importante controlar el acceso a los datos?

Porque algunos datos necesitan estar protegidos.

Por ejemplo, en una cuenta bancaria no sería conveniente que cualquier parte del programa pudiera cambiar directamente el saldo.

El acceso controlado permite establecer reglas para modificar la información.

18. ¿Qué problemas pueden presentarse si cualquier parte del programa puede modificar directamente los datos?

Podrían ocurrir errores como:

- Cambiar valores incorrectamente.
- Perder información.
- Modificar datos sin cumplir condiciones.
- Generar resultados incorrectos.

- Dañar la lógica del programa.

19. ¿Cómo se puede implementar encapsulamiento en JavaScript?

Una forma de implementarlo en JavaScript es utilizando propiedades privadas con el símbolo #.

Ejemplo:

```
class Cuenta {  
    #saldo;  
  
    constructor(saldo) {  
        this.#saldo = saldo;  
    }  
  
    consultarSaldo() {  
        return this.#saldo;  
    }  
}
```

La propiedad #saldo solamente puede utilizarse directamente dentro de la clase.

20. ¿Qué significa utilizar # para declarar una propiedad?

El símbolo # indica que una propiedad es **privada**.

Ejemplo:

```
#saldo;
```

Esto significa que el acceso directo a esa propiedad está restringido desde fuera de la clase.

21. ¿Qué ventajas tiene utilizar propiedades privadas?

Algunas ventajas son:

- Protegen la información.
- Evitan modificaciones directas.
- Permiten controlar el acceso.
- Ayudan a mantener la seguridad de los datos.
- Mejoran la organización del código.

ANÁLISIS DE LA SITUACIÓN

¿Debería cualquier parte del programa modificar directamente el saldo?

No.

El saldo debería estar protegido para evitar modificaciones incorrectas.

¿Qué problemas podrían ocurrir?

Podría ocurrir que una parte del programa cambie el saldo sin realizar correctamente una consignación o un retiro.

Por ejemplo, podría cambiarse accidentalmente de:

\$500.000

a:

\$50.000

sin realizar una operación bancaria válida.

¿Cómo podría protegerse el saldo?

Se puede utilizar una propiedad privada:

#saldo;

¿Qué métodos podrían utilizarse para modificarlo?

Se pueden utilizar métodos como:

consignar()

y:

retirar()

De esta manera, la modificación del saldo se realiza mediante métodos controlados.

ACTIVIDAD 4 – CUADRO COMPARATIVO

Concepto	¿Qué es?	¿Para qué sirve?	Ejemplo de la vida real	Ejemplo en JavaScript
Herencia	Permite que una clase reciba características de otra clase.	Para reutilizar código y compartir características.	Un perro pertenece al grupo de los animales.	<code>class Perro extends Animal {}</code>
Polimorfismo	Permite que un mismo método tenga diferentes comportamientos.	Para que cada objeto pueda realizar una acción de forma diferente.	Un carro, una moto y un bus se pueden mover, pero cada uno lo hace de manera diferente.	<code>hacerSonido() puede producir resultados diferentes.</code>
Encapsulamiento	Protege los datos internos de una clase.	Para controlar cómo se consulta o modifica la información.	El saldo de una cuenta bancaria no debe ser modificado directamente.	<code>#saldo</code>

ACTIVIDAD 5 – EJEMPLO DE LA VIDA REAL

Sistema de vehículos

Para esta actividad seleccionaremos un **sistema de vehículos**, una de las situaciones propuestas en el documento.

Podemos tener una clase general llamada:

Vehiculo

Y diferentes clases que heredan de ella:

Vehiculo

/ \

Carro Motocicleta

Herencia: ¿Quién sería la clase padre y cuáles serían las clases hijas?

La clase padre sería:

Vehiculo

Las clases hijas podrían ser:

Carro

Motocicleta

La clase Vehiculo podría tener características comunes como:

- Marca.
- Modelo.
- Método para avanzar.

Las clases hijas tendrían características propias.

Polimorfismo: ¿Qué comportamiento podría cambiar dependiendo del objeto?

Podemos tener un método llamado:

moverse()

Un carro podría mostrar:

El carro está avanzando por la carretera.

Mientras que una motocicleta podría mostrar:

La motocicleta está avanzando.

El método tiene el mismo nombre, pero cambia su comportamiento.

Encapsulamiento: ¿Qué información debería estar protegida?

Podríamos proteger información como el kilometraje del vehículo.

Por ejemplo:

#kilometraje;

Así se evita que cualquier parte del programa modifique directamente ese dato.

ACTIVIDAD 6 – EJERCICIO PRÁCTICO EN JAVASCRIPT

```
1  class Vehiculo {
2      #kilometraje;
3
4      constructor(marca, modelo, kilometraje) {
5          this.marca = marca;
6          this.modelo = modelo;
7          this.#kilometraje = kilometraje;
8      }
9
10     consultarKilometraje() {
11         return this.#kilometraje;
12     }
13
14     aumentarKilometraje(kilometros) {
15         if (kilometros > 0) {
16             this.#kilometraje += kilometros;
17         }
18     }
19
20     moverse() {
21         console.log("El vehículo se está moviendo.");
22     }
23 }
24
25
26 class Carro extends Vehiculo {
27
28     constructor(marca, modelo, kilometraje, puertas) {
29         super(marca, modelo, kilometraje);
30         this.puertas = puertas;
31     }
32
33     moverse() {
34         console.log("El carro está avanzando por la carretera.");
35     }
36 }
37
38
39 class Motocicleta extends Vehiculo {
40
41     constructor(marca, modelo, kilometraje, cilindrada) {
42         super(marca, modelo, kilometraje);
43         this.cilindrada = cilindrada;
44     }
45
46     moverse() {
47         console.log("La motocicleta está avanzando por la carretera.");
48     }
49 }
```

```

48     }
49 }
50
51 // Crear objetos
52
53 let carro = new Carro("Toyota", 2022, 15000, 4);
54
55 let moto = new Motocicleta("Yamaha", 2023, 8000, 150);
56
57 // Mostrar información
58
59 console.log("CARRO");
60 console.log("Marca:", carro.marca);
61 console.log("Modelo:", carro.modelo);
62 console.log("Puertas:", carro.puertas);
63
64 carro.moverse();
65
66 console.log("Kilometraje inicial:", carro.consultarKilometraje());
67
68 // Modificar el kilometraje mediante un método
69
70 carro.aumentarKilometraje(500);
71
72 console.log("Kilometraje después:", carro.consultarKilometraje());
73
74 console.log("MOTOCICLETA");
75 console.log("Marca:", moto.marca);
76 console.log("Modelo:", moto.modelo);
77 console.log("Cilindrada:", moto.cilindrada);
78
79 moto.moverse();
80
81 console.log("Kilometraje:", moto.consultarKilometraje());

```

Problems Output Debug Console Terminal Ports

[Running] node "c:\Users\sw-mañana\Downloads\trabajos.alexandra\clases\PRACTICA.JS"

CARRO

Marca: Toyota

Modelo: 2022

Puertas: 4

El carro está avanzando por la carretera.

Kilometraje inicial: 15000

Kilometraje después: 15500

MOTOCICLETA

Marca: Yamaha

Modelo: 2023

Cilindrada: 150

La motocicleta está avanzando por la carretera.

Kilometraje: 8000

[Done] exited with code=0 in 0.122 seconds

CONCLUSIONES

1. La herencia permite reutilizar características y métodos de una clase padre en diferentes clases hijas, evitando repetir código.
2. El polimorfismo permite que un mismo método tenga diferentes comportamientos dependiendo del objeto que lo utilice.
3. El encapsulamiento permite proteger los datos de una clase y controlar la forma en que estos pueden ser consultados o modificados.
4. JavaScript permite aplicar estos conceptos mediante herramientas como `extends`, `super()` y las propiedades privadas utilizando `#`.
5. Los tres conceptos permiten organizar mejor los programas y facilitan el desarrollo de aplicaciones utilizando Programación Orientada a Objetos.

FUENTES CONSULTADAS

- Servicio Nacional de Aprendizaje (SENA). Actividad Investigativa Individual: Herencia, Polimorfismo y Encapsulamiento en JavaScript. Material de formación de Programación Orientada a Objetos.
- MDN Web Docs. Classes – JavaScript. Documentación sobre clases, herencia y propiedades privadas en JavaScript. [MDN Web Docs – Classes](#)
- MDN Web Docs. Using classes – JavaScript. Documentación sobre clases, constructores, herencia y encapsulamiento en JavaScript. [MDN Web Docs – Using classes](#)
- MDN Web Docs. Private elements – JavaScript. Documentación sobre el uso de `#` para crear elementos privados en las clases de JavaScript. [MDN Web Docs – Private elements](#)